Speclint

How It Works

The System — 5 Files

```
docs/
├── SPEC_TEMPLATE.md          ← How to write a spec
├── SUB_AGENT_HARNESS.md      ← How to delegate work
├── MULTI_AGENT_PATTERNS.md   ← How agents are organized
├── AGENTS.md                 ← Living governance rules
└── (Speclint API)            ← Scores specs against the template
```

Each file has a job. Together they form a closed loop.

File 1: SPEC_TEMPLATE.md

The starting point. Every task gets written in 4 sections:

1. **Context & Intent** — What are we doing and why?
2. **Constraints** — What NOT to do (guardrails)
3. **Success Criteria** — What does "done" look like?
4. **Verification Steps** — How do you prove it worked?

Each section has a quality test. Example:

"Could this agent succeed at the task but cause damage we didn't anticipate?" → If yes, add a constraint.

How the Template Connects to Speclint

The 4 template sections map 1:1 to Speclint's scoring rubric:

Template Section	Speclint Checks	Points
Context & Intent	`has_measurable_outcome`	20
Constraints	`has_constraints`	20

File 2: SUB_AGENT_HARNESS.md

How we actually delegate work to coding agents.

Key concepts:

- Sub-agents start cold — they know nothing you don't tell them
- Build → Verify → Fix loop (agents stop at first plausible solution, force verification)
- Loop detection: tried 3 times without progress → stop

File 3: MULTI_AGENT_PATTERNS.md

How agents are organized. Two tiers:

Orchestrator (main session)

- Holds business context (goals, strategy, memory)
- Writes specs, lints through Speclint
- Monitors and steers sub-agents

The 4 Patterns

Fan-Out: Orchestrator spawns N parallel agents → each completes independently → consolidate

Pipeline: Agent A output → becomes Agent B input → becomes Agent C input

Coordinator + Gate: Speclint scores the spec → `agent_ready: true` gates the spawn

Watchdog: For long tasks, check in every 30 min against success criteria (not "are you done?")

File 4: AGENTS.md

The living governance document. Updated on every mistake.

What's in it:

- Mandatory pre-flight checklist (SPEC_TEMPLATE.md sections filled?)
- Dogfood rule: lint through Speclint before every spawn
- Model selection by task complexity (local Ollama → Sonnet → Opus)
- Sprint execution rules

The Full Flow

```
1. Human gives a task
          ↓
2. Orchestrator writes spec (SPEC_TEMPLATE.md)
          ↓
3. Spec goes through Speclint API
          ↓
4. agent_ready: false? → Refine, re-lint
   agent_ready: true?  → Continue
          ↓
5. Spawn sub-agent (SUB_AGENT_HARNESS.md rules)
          ↓
6. Agent executes (MULTI_AGENT_PATTERNS.md)
          ↓
7. Verify against success criteria
          ↓
8. If failure → Harness Loop (AGENTS.md updated)
```

Where Speclint Fits for Others

You write specs by hand?

→ Speclint scores them. CLI or API. 4 seconds, $0.001.

You use GitHub Issues?

→ GitHub Action auto-lints every new issue. Published on Marketplace.

You run an orchestration system?

→ API call in your pipeline: `agent_ready` boolean gates

That's the system.

Five files. One scoring rubric. Closed loop.